Construindo um Sensor de Tráfego Industrial

Modbus TCP e S7comm decodificados à mão, em Python, pensando no C

Gabriel de Oliveira e Silva
Monitoria de Introdução às Redes de Comunicação, UFPE

2026.2

SNIFFER

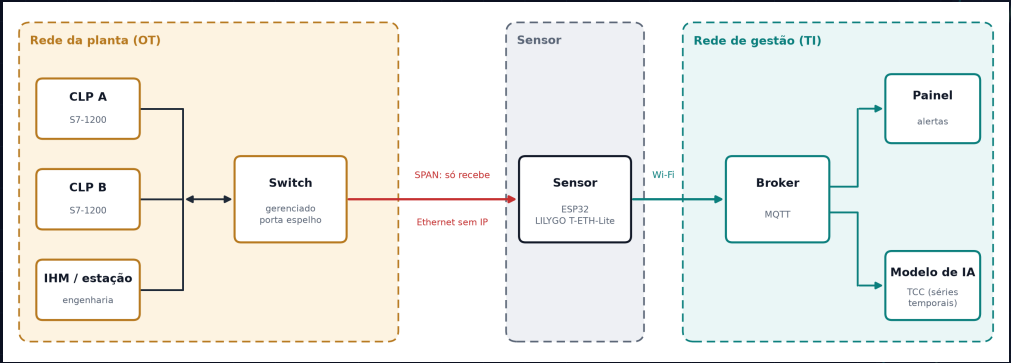
De onde viemos

- **Aula 1:** Modbus TCP e Wireshark
- **Aula 2:** o CLP como servidor Modbus
- **PUT/GET:** dois CLPs conversando por S7
- **Switch:** espelhamento de porta, VLAN

Hoje

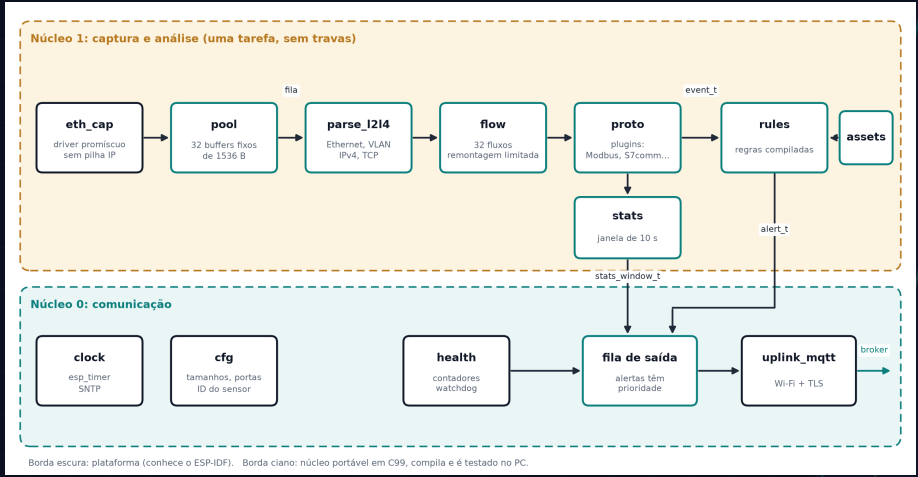
O Wireshark mostra os pacotes, mas alguém precisa olhar. Vamos escrever o programa que olha por nós: lê cada pacote, entende o protocolo e decide se é normal.

O que vamos construir: um sensor passivo

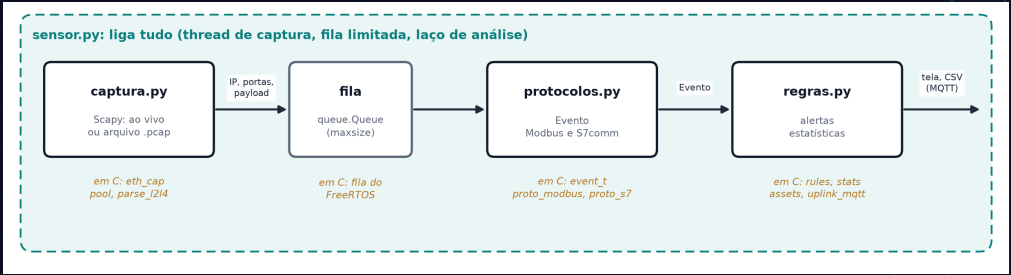


O sensor só escuta: não tem IP na rede da planta e nunca transmite nela.

O projeto real: um sensor em C no ESP32



Hoje: as mesmas ideias em 4 arquivos de Python



1 capturar 2 Modbus 3 S7comm 4 regras e sensor

Preparação

Na pasta da aula já tem:

- `exemplo.pcap`: tráfego da bancada, com um intruso
- `carga.pcap`: 100 mil pacotes
- `ferramentas/`: gerador de tráfego para o CLP

Vocês criam: `captura.py`, `protocolos.py`, `regras.py`, `sensor.py`, na mesma pasta do `exemplo.pcap`.

No terminal, na pasta da aula:

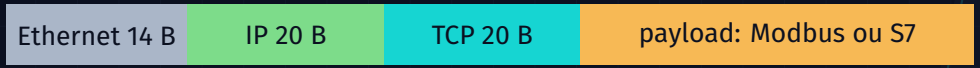
```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
pip install scapy
```

Captura ao vivo no Windows: Npcap instalado e terminal como administrador. Com arquivo .pcap, não precisa.

Etapa 1: do quadro ao segmento



é isso que vamos guardar no `Segmento`

- O Scapy separa as camadas: `pacote[IP]`, `pacote[TCP]`
- Do IP: endereços. Do TCP: portas. O resto é o payload
- Quadros pequenos ganham enchimento da Ethernet: o tamanho certo vem do IP

Etapa 1: o Segmento

DIGITE em captura.py (arquivo novo)

```
from dataclasses import dataclass

from scapy.all import IP, TCP, PcapReader, sniff

PORTAS = {502, 102}   # 502 = Modbus TCP, 102 = S7comm


@dataclass
class Segmento:
    ts: float          # hora da captura, em segundos
    ip_origem: str
    ip_destino: str
    porta_origem: int
    porta_destino: int
    payload: bytes     # os dados que o TCP carrega
```

Etapa 1: do pacote do Scapy ao Segmento

DIGITE em `captura.py` (continue embaixo)

```
def para_segmento(pacote):
    if IP not in pacote or TCP not in pacote:
        return None
    ip, tcp = pacote[IP], pacote[TCP]
    # tamanho dos dados = total do IP - cabeçalho IP - cabeçalho TCP
    tamanho = ip.len - ip.ihl * 4 - tcp.dataofs * 4
    if tamanho <= 0:
        return None    # SYN, ACK puro, FIN: sem dados
    dados = bytes(tcp.payload)[:tamanho]
    return Segmento(float(pacote.time), ip.src, ip.dst,
                    tcp.sport, tcp.dport, dados)
```

`ihl` e `dataofs` contam em palavras de 4 bytes: por isso o `* 4`.

Etapa 1: ler um arquivo ou capturar ao vivo

DIGITE em `captura.py` (continue embaixo)

```
def ler_arquivo(caminho):
    for pacote in PcapReader(caminho):
        seg = para_segmento(pacote)
        if seg and (seg.porta_origem in PORTAS
                    or seg.porta_destino in PORTAS):
            yield seg


def capturar(interface, ao_receber):
    def tratar(pacote):
        seg = para_segmento(pacote)
        if seg:
            ao_receber(seg)
    sniff(iface=interface, filter="tcp port 502 or tcp port 102",
          prn=tratar, store=False)
```

Etapa 1: teste

DIGITE em `captura.py` (no fim do arquivo)

```
if __name__ == "__main__":
    for seg in ler_arquivo("exemplo.pcap"):
        print(seg.ip_origem, "->", seg.ip_destino,
              seg.porta_destino, seg.payload.hex(" "))
```

RODE

```
python captura.py
```

```
192.168.0.29 -> 192.168.0.11 502 00 01 00 00 00 06 01 03 00 00 00 0a

192.168.0.11 -> 192.168.0.12 102 03 00 00 1f 02 f0 80 32 01 00 00 00 01 00 0e ...
```

Em C: o mesmo trabalho, sem Scapy

EM C, NO ESP32

```c
/* parse_l2l4: o driver entrega o quadro cru; cada cabeçalho é conferido à mão */
int parse_l2l4(const uint8_t *q, size_t len, frame_info_t *fi) {
    if (len < 14 + 20 + 20) return -1;            /* pequeno demais        */
    if (rd16(q + 12) != 0x0800) return -1;        /* não é IPv4            */
    const uint8_t *ip = q + 14;
    size_t ihl = (ip[0] & 0x0F) * 4;              /* o mesmo "* 4"         */
    if (ip[9] != 6) return -1;                    /* protocolo 6 = TCP     */
    const uint8_t *tcp = ip + ihl;
    size_t doff = (tcp[12] >> 4) * 4;
    size_t total = rd16(ip + 2);                  /* ip.len                */
    if (total > len - 14 || ihl + doff > total) return -1;
    fi->src_ip = rd32(ip + 12);  fi->dst_ip = rd32(ip + 16);
    fi->src_port = rd16(tcp);    fi->dst_port = rd16(tcp + 2);
    fi->payload = tcp + doff;                     /* aponta, não copia     */
    fi->payload_len = total - ihl - doff;
    return 0;
}
```

Etapa 2: os bytes de uma pergunta Modbus TCP

```
00 01   00 00   00 06   01   03   00 00   00 0a
```

Bytes	Campo	Valor	Para que serve
`00 01`	Transaction ID	1	casar pergunta e resposta
`00 00`	Protocol ID	sempre 0	senão, não é Modbus
`00 06`	Length	6 bytes depois deste campo	a mensagem está inteira?
`01`	Unit ID	1	
`03`	Função	3 = ler holding registers	a operação
`00 00  00 0a`	Dados da função	endereço 0, quantidade 10	

Os 7 primeiros bytes são o cabeçalho MBAP. Todo número é big-endian: `00 0a` = 10.

Etapa 2: o Evento

DIGITE em `protocolos.py` (arquivo novo)

```python
import struct
from dataclasses import dataclass, field


@dataclass
class Evento:
    ts: float
    origem: str
    destino: str
    protocolo: str      # "modbus" ou "s7"
    direcao: str        # "pergunta" ou "resposta"
    operacao: str       # "leitura", "escrita", "controle", "outra"
    funcao: int         # código da função no protocolo
    area: str = ""
    endereco: int = 0
    quantidade: int = 0
    status: int = 0     # 0 = ok; senão, código de erro
    valores: list = field(default_factory=list)
```

Etapa 2: como o Evento aparece no print

DIGITE em `protocolos.py` (dentro da classe Evento, indentado)

```
    def __str__(self):
        return (f"{self.origem:>15} -> {self.destino:<15} "
                f"{self.protocolo:6} {self.direcao:8} {self.operacao:9}"
                f" f={self.funcao:#04x} {self.area} end={self.endereco}"
                f" qtd={self.quantidade} {self.valores}"
                f" status={self.status:#06x}")
```

`{self.funcao:#04x}` mostra o número em hexadecimal: 3 vira `0x03`.

Etapa 2: perguntas pendentes e a escolha pela porta

DIGITE em `protocolos.py` (fora da classe, sem indentação)

```
PENDENTES = {}         # perguntas esperando resposta
DECODIFICADORES = {}   # porta do servidor -> função que decodifica


def palavras(dados):
    """Até 4 números de 16 bits (big-endian) do início de dados."""
    n = min(len(dados) // 2, 4)
    return list(struct.unpack(f">{n}H", dados[:n * 2]))


def decodificar(seg):
    porta = (seg.porta_destino if seg.porta_destino in DECODIFICADORES
             else seg.porta_origem)
    funcao = DECODIFICADORES.get(porta)
    return funcao(seg) if funcao else None
```

Etapa 2: o cabeçalho Modbus

DIGITE em `protocolos.py` (continue embaixo)

```
MODBUS_OPERACAO = {1: "leitura", 2: "leitura", 3: "leitura",
                   4: "leitura", 5: "escrita", 6: "escrita",
                   15: "escrita", 16: "escrita"}


def decodificar_modbus(seg):
    d = seg.payload
    if len(d) < 8:
        return None
    # MBAP (7 bytes) + função (1 byte), tudo big-endian (">")
    transacao, protocolo, tamanho, unidade, funcao = struct.unpack(
        ">HHHBB", d[:8])
    if protocolo != 0 or len(d) != 6 + tamanho:
        return None              # não é UMA mensagem Modbus inteira
    pergunta = seg.porta_destino == 502
    erro = funcao & 0x80         # bit 7 ligado = resposta de erro
    funcao = funcao & 0x7F
    corpo = d[8:]
```

Etapa 2: a pergunta Modbus

DIGITE em `protocolos.py` (continua dentro da função)

```
    ev = Evento(seg.ts, seg.ip_origem, seg.ip_destino, "modbus",
                "pergunta" if pergunta else "resposta",
                MODBUS_OPERACAO.get(funcao, "outra"), funcao)
    if pergunta:
        if len(corpo) >= 4:
            ev.endereco, ev.quantidade = struct.unpack(">HH", corpo[:4])
        if funcao in (5, 6):      # escrita simples: o 2o campo é o valor
            ev.valores, ev.quantidade = [ev.quantidade], 1
        PENDENTES[("modbus", seg.ip_origem, transacao)] = ev
```

Pergunta `03 00 00 00 0a`: depois da função, `00 00` = endereço 0 e `00 0a` = quantidade 10.

Etapa 2: a resposta Modbus

DIGITE em `protocolos.py` (continua dentro da função)

```
    else:
        p = PENDENTES.pop(("modbus", seg.ip_destino, transacao), None)
        if p:                       # a resposta não repete o endereço
            ev.endereco, ev.quantidade = p.endereco, p.quantidade
        if erro:
            ev.status = corpo[0]
        elif funcao in (3, 4):
            ev.valores = palavras(corpo[1:])
    return ev


DECODIFICADORES[502] = decodificar_modbus
```

Resposta `03 14 00 fa ...`: `14` = 20 bytes de dados; `00 fa` = 250, o primeiro registrador.

Etapa 2: teste

DIGITE em `protocolos.py` (no fim do arquivo)

```
if __name__ == "__main__":
    from captura import ler_arquivo
    for seg in ler_arquivo("exemplo.pcap"):
        print(decodificar(seg))
```

RODE

```
python protocolos.py
```

```
192.168.0.29 -> 192.168.0.11  modbus pergunta leitura f=0x03  end=0 qtd=10 []
status=0x0000
192.168.0.11 -> 192.168.0.29  modbus resposta leitura f=0x03  end=0 qtd=10 [250, 1,
0, 0] ...
None
```

Em C: o plugin de protocolo

EM C, NO ESP32

```c
typedef struct proto {
    const char *name;
    uint16_t    tcp_port;                    /* 502, 102                    */
    /* tamanho da próxima mensagem: >0 bytes, 0 falta chegar, <0 lixo      */
    int  (*frame_len)(const uint8_t *buf, size_t len);
    /* decodifica UMA mensagem inteira e preenche o evento                 */
    int  (*decode)(const uint8_t *msg, size_t len, msg_dir_t dir,
                   proto_state_t *st, event_t *out);
} proto_t;

static const proto_t *const PROTOS[] = { &PROTO_MODBUS, &PROTO_S7 };

static int modbus_frame_len(const uint8_t *b, size_t n) {
    if (n < 6) return 0;                        /* falta o MBAP          */
    if (rd16(b + 2) != 0) return -1;            /* Protocol ID != 0      */
    return 6 + rd16(b + 4);                     /* o mesmo 6 + tamanho   */
}
```

Intervalo

10 minutos

Quem não chegou no teste da etapa 2: copie
`codigo/protocolos.py`
até a linha do `DECODIFICADORES[502]` e o bloco de teste.

Etapa 3: S7comm vem dentro de duas caixas

```
03 00 00 1f  02 f0 80  32 01 00 00 00 01 00 0e 00 00
04 01 12 0a 10 02 00 14 00 01 84 00 00 00
```

Camada	**Bytes**	O que importa
TPKT (4 B)	`03 00 00 1f`	versão 3; tamanho total 0x1f = 31 bytes
COTP (3 B)	`02 f0 80`	`02` = tamanho do resto do COTP; `f0` = dados
S7, cabeçalho (10 B)	`32 01 00 00 00 01 00 0e 00 00`	`32` = S7; `01` = pedido; PDU ref 1; 14 B de parâmetros; 0 de dados
S7, parâmetros	`04 01 12 0a ...`	função 4 (Read Var), 1 item

Na resposta, o cabeçalho S7 tem 12 bytes: os 2 a mais são o código de erro.

Etapa 3: o item de leitura (12 bytes)

```
12  0a  10  02  00 14  00 01  84  00 00 00
```

Bytes	Campo	Valor	Em **param**
`12 0a 10`	início do item, tamanho, sintaxe		2 a 4
`02`	tipo de dado	BYTE	5
`00 14`	quantidade	20	6 e 7
`00 01`	número do DB	DB1	8 e 9
`84`	área	0x84 = DB (0x81 entradas, 0x82 saídas, 0x83 memórias)	10
`00 00 00`	endereço, em bits	0 (byte = bits ÷ 8, ou » 3)	11 a 13

É o `P#DB1.DBX0.0 BYTE 20` do bloco GET, em bytes.

Etapa 3: atravessando TPKT e COTP

DIGITE em `protocolos.py` (antes do bloco `if __name__`)

```
S7_OPERACAO = {0x04: "leitura", 0x05: "escrita",
               0x28: "controle", 0x29: "controle"}


def decodificar_s7(seg):
    d = seg.payload
    # TPKT: versão 3 e tamanho total igual ao que chegou
    if len(d) < 17 or d[0] != 3:
        return None
    if struct.unpack(">H", d[2:4])[0] != len(d):
        return None
    if d[5] != 0xF0:              # COTP de dados; senão é conexão
        return None
    s7 = d[5 + d[4]:]             # pula TPKT (4) e COTP (1 + d[4])
    rosctr = s7[1]                # 1 = pedido; 2 e 3 = resposta
    if s7[0] != 0x32 or rosctr not in (1, 2, 3):
        return None
```

Etapa 3: o cabeçalho S7

DIGITE em `protocolos.py` (continua dentro da função)

```
    pdu_ref, tam_param, tam_dados = struct.unpack(">HHH", s7[4:10])
    pergunta = rosctr == 1
    inicio = 10 if pergunta else 12   # resposta: +2 bytes de erro
    if len(s7) < inicio:
        return None
    param = s7[inicio:inicio + tam_param]
    dados = s7[inicio + tam_param:inicio + tam_param + tam_dados]
    funcao = param[0] if param else 0
    ev = Evento(seg.ts, seg.ip_origem, seg.ip_destino, "s7",
                "pergunta" if pergunta else "resposta",
                S7_OPERACAO.get(funcao, "outra"), funcao)
```

Etapa 3: a pergunta S7

DIGITE em `protocolos.py` (continua dentro da função)

```
    if pergunta:
        # Read/Write Var: o 1o item começa em param[2], 12 bytes
        if funcao in (0x04, 0x05) and len(param) >= 14:
            ev.quantidade, db = struct.unpack(">HH", param[6:10])
            ev.area = f"DB{db}" if param[10] == 0x84 else hex(param[10])
            ev.endereco = int.from_bytes(param[11:14], "big") >> 3
        if funcao == 0x05:
            ev.valores = palavras(dados[4:])
        PENDENTES[("s7", seg.ip_origem, pdu_ref)] = ev
```

As posições `param[6:10]`, `param[10]` e `param[11:14]` são as da tabela do item de 12 bytes.

Etapa 3: a resposta S7

DIGITE em `protocolos.py` (continua dentro da função)

```
    else:
        p = PENDENTES.pop(("s7", seg.ip_destino, pdu_ref), None)
        if p:
            ev.funcao, ev.operacao, ev.area = p.funcao, p.operacao, p.area
            ev.endereco, ev.quantidade = p.endereco, p.quantidade
        ev.status = struct.unpack(">H", s7[10:12])[0]   # ex.: 0x8104
        if ev.status == 0 and dados and dados[0] != 0xFF:
            ev.status = dados[0]    # erro do item: 0x0A = não existe
        elif ev.status == 0 and ev.funcao == 0x04:
            ev.valores = palavras(dados[4:])
    return ev

DECODIFICADORES[102] = decodificar_s7
```

Etapa 3: teste

RODE

```
python protocolos.py
```

```
192.168.0.11 -> 192.168.0.12 s7     resposta leitura f=0x04 DB1 end=0 qtd=20 [82, 0, 0, 0]
...
192.168.0.50 -> 192.168.0.11 modbus pergunta escrita f=0x06 end=10 qtd=1 [1500]
192.168.0.11 -> 192.168.0.50 s7     resposta leitura f=0x04 DB1 status=0x8104

192.168.0.50 -> 192.168.0.12 s7     pergunta controle f=0x29
```

Achou o intruso?

Alguém em `192.168.0.50` escreveu 1500 num registrador, tentou ler o CLP por S7 e mandou um PLC STOP (função `0x29`).

Etapa 4: o que é suspeito numa rede de CLPs?

Regra	Dispara quando	Por quê
R01 ALTA	escrita vinda de quem não pode escrever	só engenharia e CLPs mudam valores
R02 CRÍTICA	comando de controle (stop, start)	parar um CLP para o processo
R03 ALTA	IP fora do inventário	numa planta, todo equipamento é conhecido
R04 MÉDIA	resposta de erro	acesso negado ou varredura de endereços

Rede industrial é previsível: sempre os mesmos equipamentos, as mesmas leituras, no mesmo ritmo. Por isso regras simples já pegam muito.

Etapa 4: o inventário e a primeira regra

DIGITE em `regras.py` (arquivo novo)

```
from collections import Counter

# Inventário da bancada: IP -> papel. Ajuste para o laboratório.
ATIVOS = {"192.168.0.11": "clp", "192.168.0.12": "clp",
          "192.168.0.29": "engenharia"}
PODEM_ESCREVER = {"clp", "engenharia"}


def escrita_nao_autorizada(ev):
    return (ev.direcao == "pergunta" and ev.operacao == "escrita"
            and ATIVOS.get(ev.origem) not in PODEM_ESCREVER)
```

Etapa 4: as outras regras

DIGITE em `regras.py` (continue embaixo)

```
def controle_do_clp(ev):
    return ev.direcao == "pergunta" and ev.operacao == "controle"


def ip_desconhecido(ev):
    return ev.origem not in ATIVOS


def resposta_de_erro(ev):
    return ev.direcao == "resposta" and ev.status != 0
```

Etapa 4: a tabela de regras

DIGITE em `regras.py` (continue embaixo)

```
REGRAS = [
    ("R01", "ALTA", "escrita de quem não pode escrever",
     escrita_nao_autorizada),
    ("R02", "CRITICA", "comando de controle no CLP", controle_do_clp),
    ("R03", "ALTA", "IP fora do inventário", ip_desconhecido),
    ("R04", "MEDIA", "resposta de erro", resposta_de_erro),
]


def avaliar(ev):
    alertas = []
    for codigo, nivel, texto, regra in REGRAS:
        if regra(ev):
            alertas.append((codigo, nivel, texto))
    return alertas
```

Etapa 4: a janela de estatísticas

DIGITE em `regras.py` (continue embaixo)

```
class Janela:
    """Conta eventos por (origem, destino, protocolo, operação)."""

    def __init__(self, segundos=10):
        self.segundos = segundos
        self.inicio = None
        self.contagem = Counter()

    def adicionar(self, ev):
        resumo = None
        if self.inicio is None:
            self.inicio = ev.ts
        elif ev.ts - self.inicio >= self.segundos:
            resumo = self.contagem          # a janela acabou
            self.inicio, self.contagem = ev.ts, Counter()
        chave = (ev.origem, ev.destino, ev.protocolo, ev.operacao)
        self.contagem[chave] += 1
        return resumo
```

Etapa 4: teste

DIGITE em `regras.py` (no fim do arquivo)

```
if __name__ == "__main__":
    from captura import ler_arquivo
    from protocolos import decodificar
    for seg in ler_arquivo("exemplo.pcap"):
        ev = decodificar(seg)
        if ev:
            for alerta in avaliar(ev):
                print(alerta, ev)
```

RODE

```
python regras.py
```

```
('R01', 'ALTA', 'escrita de quem não pode escrever')  192.168.0.50 -> 192.168.0.11 ...
('R04', 'MEDIA', 'resposta de erro')  192.168.0.11 -> 192.168.0.50 ...  status=0x8104
```

Em C: regras são dados, não código

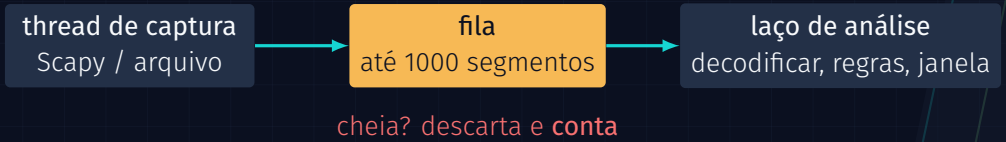
EM C, NO ESP32

```c
typedef struct {
    const char *id;     uint8_t severity;
    uint8_t  proto_id;  uint8_t op;        /* ANY = qualquer          */
    uint8_t  src_role_not;                 /* dispara se a origem NÃO */
    uint16_t rate_max_per_s;               /* 0 = sem limite de taxa  */
} rule_t;

static const rule_t RULES[] = {            /* gravado no firmware     */
    { "R01", SEV_HIGH, ANY, OP_WRITE,   ROLE_ENG | ROLE_PLC, 0 },
    { "R02", SEV_CRIT, ANY, OP_CONTROL, ROLE_NONE,           0 },
    { "R04", SEV_MED,  ANY, OP_ANY,     ROLE_NONE,          50 },
};
```

Regras compiladas: ninguém consegue mudar a detecção pela rede.

Etapa 5: duas tarefas e uma fila



- A placa de rede não espera: se o sensor demora, os pacotes chegam do mesmo jeito
- A fila absorve picos; se mesmo assim encher, o sensor perde e avisa
- No ESP32: duas tarefas do FreeRTOS, em dois núcleos, e uma `xQueue`

Etapa 5: importações e a fila

DIGITE em `sensor.py` (arquivo novo)

```
import argparse
import queue
import threading
import time
from collections import Counter

from captura import capturar, ler_arquivo
from protocolos import decodificar
from regras import Janela, avaliar

contadores = Counter()
fila = queue.Queue(maxsize=1000)    # em C: xQueueCreate(1000, ...)
```

Etapa 5: colocar na fila

DIGITE em `sensor.py` (continue embaixo)

```
def ao_receber(seg):                # roda na thread de captura
    contadores["capturados"] += 1
    try:
        fila.put_nowait(seg)
    except queue.Full:              # fila cheia: descarta e conta
        contadores["descartados"] += 1
```

Etapa 5: iniciar a captura

DIGITE em `sensor.py` (continue embaixo)

```
def do_arquivo(caminho):
    for seg in ler_arquivo(caminho):
        contadores["capturados"] += 1
        fila.put(seg)                # do arquivo: espera, não descarta
    fila.put(None)                   # aviso de fim


def iniciar(args):
    if args.arquivo:
        alvo, parametros = do_arquivo, (args.arquivo,)
    else:
        alvo, parametros = capturar, (args.interface, ao_receber)
    threading.Thread(target=alvo, args=parametros, daemon=True).start()
```

Etapa 5: analisar cada evento

DIGITE em `sensor.py` (continue embaixo)

```
def analisar(ev, janela, quieto):
    if not quieto:
        print(ev)
    for codigo, nivel, texto in avaliar(ev):
        contadores["alertas"] += 1
        print(f"  !!! {codigo} {nivel}: {texto}")
        print(f"      {ev}")
    resumo = janela.adicionar(ev)
    if resumo and not quieto:
        print("--- últimos 10 s:")
        for chave, n in resumo.items():
            print("   ", *chave, n)
```

Etapa 5: o programa principal

DIGITE em `sensor.py` (continue embaixo)

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--interface")
    p.add_argument("--arquivo")
    p.add_argument("--quieto", action="store_true")
    args = p.parse_args()
    iniciar(args)
    janela, inicio = Janela(), time.perf_counter()
```

Exemplos: `--arquivo exemplo.pcap`, `--interface "Ethernet"`, `--quieto`.

Etapa 5: o laço de análise

DIGITE em `sensor.py` (continua dentro do `main`)

```
    try:
        while True:
            try:
                seg = fila.get(timeout=1)   # timeout: deixa o Ctrl+C agir
            except queue.Empty:
                continue
            if seg is None:
                break
            ev = decodificar(seg)
            if ev is None:
                contadores["nao_decodificados"] += 1
            else:
                analisar(ev, janela, args.quieto)
    except KeyboardInterrupt:
        pass
```

Etapa 5: o fim, e rodar o sensor

DIGITE em `sensor.py` (continua dentro do `main`)

```
    segundos = time.perf_counter() - inicio
    print(dict(contadores))
    print(f"{contadores['capturados'] / segundos:.0f} segmentos/s")

if __name__ == "__main__":
    main()
```

RODE

```
python sensor.py --arquivo exemplo.pcap
```

```
  !!! R02 CRITICA: comando de controle no CLP
--- últimos 10 s: ...
{'capturados': 53, 'alertas': 6}
```

Ao vivo na bancada

Terminal 1, gerando tráfego com o CLP da bancada:

```
cd ferramentas
```

```
python gerar_trafego.py --clp 192.168.0.11
```

Terminal 2, como administrador, o sensor na placa ligada aos CLPs:

```
python sensor.py --interface "Ethernet"
```

O nome da placa aparece em `Get-NetAdapter`. O gerador só lê; a escrita do "intruso" é feita pelo monitor.

O limite do Python

RODE

```
python sensor.py --arquivo carga.pcap --quieto
```

Nosso sensor

$\sim$2.500

segmentos por segundo

Uma rede de 100 Mbit

$\sim$148.000

quadros por segundo (no máximo)

Quase todo o tempo vai para o Scapy montar um objeto Python para cada pacote.

O que falta para embarcar (ESP32)

Hoje, em Python	No ESP32, em C	Por quê
Scapy separa as camadas	`parse_l2l4` à mão, ponteiros	sem cópia, sem objetos
memória livre	pool e tabelas fixas, nada de `malloc`	nunca travar por falta de memória
1 mensagem por pacote TCP	remontagem limitada (1 KB por sentido)	mensagem pode vir em pedaços
uma thread, um núcleo de verdade	2 tarefas em 2 núcleos	captura e análise em paralelo
filtro BPF no Npcap	Ethernet promíscua sem IP	passivo de verdade
`print` e CSV	MQTT com TLS pelo Wi-Fi	entregar para painel e IA

O que falta para instalar numa planta real

- **TAP passivo** em vez de porta espelho: o espelho perde quadros sob carga
- **Inventário** levantado com a operação
- **Regras compiladas** e firmware assinado
- **TLS** no MQTT: os valores do processo são sensíveis

- **Falso positivo:** quem recebe o alerta, e o que faz com ele?
- **Saúde do sensor:** contadores de descarte publicados
- **Limites:** S7comm-plus (TIA Portal) e tráfego com TLS não são decodificados, só contados

Do sensor ao meu TCC

Detecção de intrusão em plantas industriais

- As janelas de 10 s viram séries temporais
- Um modelo de IA aprende o comportamento normal
- Regras físicas da planta conferem o que o modelo diz
- Objetivo: separar defeito elétrico, instrumento descalibrado e ataque

Projeto de pesquisa

Validação nas bases públicas **HAI** e **SWaT**, e na bancada do laboratório.

Quem quiser participar: fale comigo no fim da aula.

Resumo

- Um sensor passivo escuta sem interferir na planta
- `struct.unpack` transforma bytes em campos: Modbus e S7 lidos à mão
- Um formato único (`Evento`) deixa as regras valerem para qualquer protocolo
- Rede industrial é previsível: regras simples já acham o intruso
- Python serve para entender e prototipar; um sensor real é C em hardware dedicado

Código completo, arquitetura em C e decisões: pasta `Projeto_Sniffer_IDS` do repositório.